

NutriLog — веб та mobile платформа для відстеження харчування, КБЖУ та мікроелементів

1. Загальна інформація про проєкт

Назва проєкту

NutriLog

Тип проєкту

Full-stack SaaS веб-платформа з подальшим масштабуванням у мобільний застосунок.

Концепція проєкту

NutriLog — це сервіс для відстеження харчування, розрахунку калорійності, КБЖУ (білки, жири, вуглеводи), мікроелементів, аналітики прогресу та персональних рекомендацій.

Користувач має змогу:

- розрахувати свою денну норму калорій та КБЖУ;
- вести щоденник харчування;
- шукати продукти та додавати їх у прийоми їжі;
- відстежувати споживання макро- та мікроелементів;
- переглядати аналітику;
- користуватися premium-функціями.

Проєкт повинен мати архітектуру, придатну для масштабування та подальшого запуску mobile-клієнта.

2. Цілі проєкту

Основна ціль

Створити сучасний аналог застосунків для підрахунку калорій (наприклад, “Таблиця калорійності”, MyFitnessPal, FatSecret), адаптований під власну архітектуру та можливість масштабування.

Secondary goals

Для користувача

- зручне ведення харчового щоденника;
- простий пошук продуктів;
- точний розрахунок КБЖУ;
- відстеження мікроелементів;
- прогрес у зміні ваги.

Для портфоліо

Проект має демонструвати:

- real-world full-stack architecture;
 - authentication & authorization;
 - OAuth;
 - subscription system;
 - REST API;
 - Redis caching;
 - PostgreSQL relations;
 - state management;
 - modular architecture;
 - responsive UI;
 - mobile-ready backend.
-

3. Технологічний стек

Frontend

Основний стек

- Next.js (App Router)
- TypeScript
- UnoCSS
- Zustand
- Axios
- React Hook Form
- Zod

- TanStack Query

UI

- Responsive design
 - Mobile-first approach
 - Light / Dark theme (у перспективі)
-

Backend

Основний стек

- NestJS
- TypeScript
- REST API
- JWT
- OAuth 2.0
- Redis
- PostgreSQL
- Prisma ORM
- Swagger

Authentication

- JWT access token
 - Refresh token
 - Google OAuth
-

Database

СУБД

PostgreSQL

ORM

Prisma

Кешування

Redis

Використання Redis:

- кешування продуктів;
 - популярні пошукові запити;
 - rate limiting;
 - session/token invalidation;
 - cache layer для frequently requested data.
-

Mobile (Future)

Framework

React Native + Expo

Причина вибору

- reuse React knowledge;
 - можливість повторного використання бізнес-логіки;
 - швидший development;
 - спільні типи та API.
-

4. Архітектура проєкту

Архітектурний підхід

Modular Monolith

Мета:

- швидша розробка;
 - простіша підтримка;
 - можливість масштабування;
 - легке винесення окремих модулів у microservices у майбутньому.
-

Backend architecture

src/

modules/

auth/

users/

nutrition/
foods/
meals/
diary/
analytics/
subscription/
payments/
notifications/
admin/

common/
config/
prisma/
shared/

Основні модулі

auth

Функціонал:

- registration
- login
- logout
- JWT
- refresh tokens
- OAuth
- email verification (future)

users

Функціонал:

- профіль користувача
- цілі
- налаштування

nutrition

Функціонал:

- розрахунок BMR
- TDEE
- добова норма КБЖУ

foods

Функціонал:

- база продуктів

- пошук
- категорії
- custom foods

meals

Функціонал:

- прийоми їжі
- додавання продуктів
- розрахунок сумарних значень

diary

Функціонал:

- денний журнал харчування
- історія

analytics

Функціонал:

- статистика
- графіки
- прогрес

subscription

Функціонал:

- premium access
- permissions

payments

Функціонал:

- Stripe
- payment history

admin

Функціонал:

- moderation foods
 - analytics
 - reports
-

Frontend architecture

Підхід: **Feature-Sliced Design (FSD inspired)**

src/

app/
features/
entities/
widgets/
shared/

features

Бізнес-функціональність:

features/
auth/
add-food/
diary/
profile/
analytics/

entities

Domain models:

entities/
user/
food/
meal/
nutrition/

widgets

Reusable sections.

shared

- api
 - hooks
 - ui
 - constants
 - utils
-

5. API architecture

API type

REST API

Причини вибору

- простіша реалізація;
- зрозумілий CRUD;
- простіше дебажити;
- mobile-friendly;
- зручніше кешувати.

Приклад endpoint structure

POST /auth/register

POST /auth/login

POST /auth/refresh

GET /foods/search?q=banana

GET /foods/:id

POST /meals

PATCH /meals/:id

DELETE /meals/:id

GET /diary/today

GET /diary/history

GET /analytics/week

6. Ролі користувачів

User

Базовий користувач.

Premium User

Розширений доступ.

Admin

Адміністративний доступ.

7. MVP scope

MVP goal

Створити повністю робочий продукт для ведення харчового щоденника.

7.1 Authentication

Реалізувати

- registration
 - login
 - logout
 - JWT auth
 - refresh token
 - Google OAuth
-

7.2 User Profile

Дані користувача

- стать
- вік
- зріст
- вага
- активність
- ціль

Цілі

- схуднення
- підтримка ваги
- набір маси

7.3 Розрахунок КБЖУ

Формули

BMR

Mifflin-St Jeor

TDEE

Activity multiplier

Розрахунок

Система має автоматично розраховувати:

- денні калорії;
- білки;
- жири;
- вуглеводи.

7.4 Food Database

База продуктів

Поля:

id
name
brand
category
calories
protein
fat
carbs
fiber
sugar
serving_size

Значення зберігаються на 100 г продукту.

7.5 Пошук продуктів

Функціонал:

- live search;
 - пошук по назві;
 - популярні продукти;
 - кешування результатів.
-

7.6 Meals

Типи прийомів їжі:

- Breakfast
- Lunch
- Dinner
- Snack

Реалізувати

- створення прийому їжі;
 - додавання продуктів;
 - редагування;
 - видалення.
-

7.7 Daily Diary

Показувати:

- калорії;
 - білки;
 - жири;
 - вуглеводи;
 - залишок до цілі;
 - progress bars.
-

7.8 Історія

- перегляд минулих днів;
- історія харчування.

8. Future scope (V2)

Micronutrients tracking

Відстеження:

- magnesium
- potassium
- sodium
- iron
- zinc
- calcium
- vitamin A
- vitamin B
- vitamin C
- vitamin D
- vitamin E
- fiber
- omega fats

Dashboard

Приклад:

Magnesium — 64%

Potassium — 83%

Iron — 47%

Analytics

Графіки

- вага
 - калорії
 - білки
 - середні значення
 - weekly statistics
-

Body Tracking

Можливість додавання:

- вага
 - обхвати
 - фото прогресу
-

Goals system

Користувач може встановлювати:

- денну ціль білка;
 - water intake;
 - streak system.
-

Barcode Scanner

Функціонал:

- сканування штрих-коду;
 - автоматичне знаходження продукту.
-

Custom Foods

Користувач може:

- створювати власні продукти;
 - зберігати рецепти.
-

Favorite meals

Швидке додавання:

- frequently used foods;
 - saved meals.
-

AI Recommendations

Система може пропонувати:

- що доїсти;
 - чого бракує;
 - рекомендовані продукти.
-

9. Premium subscription

Free

Доступ:

- щоденник;
 - КБЖУ;
 - базова історія.
-

Premium

Доступ:

- unlimited history;
 - analytics;
 - micronutrients;
 - AI recommendations;
 - meal planning;
 - export PDF;
 - advanced tracking.
-

10. Payment system

Web

Stripe

Mobile

- Google Play Billing

- Apple In-App Purchase
-

11. Non-functional requirements

Security

- password hashing;
 - refresh tokens;
 - secure cookies;
 - rate limiting;
 - validation;
 - DTO validation;
 - RBAC.
-

Performance

- Redis caching;
 - DB indexes;
 - pagination;
 - optimized queries.
-

Scalability

Архітектура повинна дозволяти:

- mobile client;
 - multiple APIs;
 - external integrations;
 - microservices migration.
-

Responsive

Сайт повинен коректно працювати:

- desktop;
- tablet;

- mobile.
-

12. Орієнтовний roadmap

Stage 1 — Foundation

- setup architecture
- auth
- database
- user profile

Stage 2 — Core nutrition

- food database
- search
- meals
- diary
- calculations

Stage 3 — Analytics

- history
- statistics
- charts

Stage 4 — Premium

- subscriptions
- Stripe
- permissions

Stage 5 — Mobile

- React Native
 - Expo
 - synchronization
-

13. Success criteria

MVP вважається завершеним якщо:

- користувач може зареєструватися;
- пройти авторизацію;
- отримати персональну норму КБЖУ;
- додавати продукти;
- вести денний журнал;
- переглядати історію;
- бачити прогрес по КБЖУ.